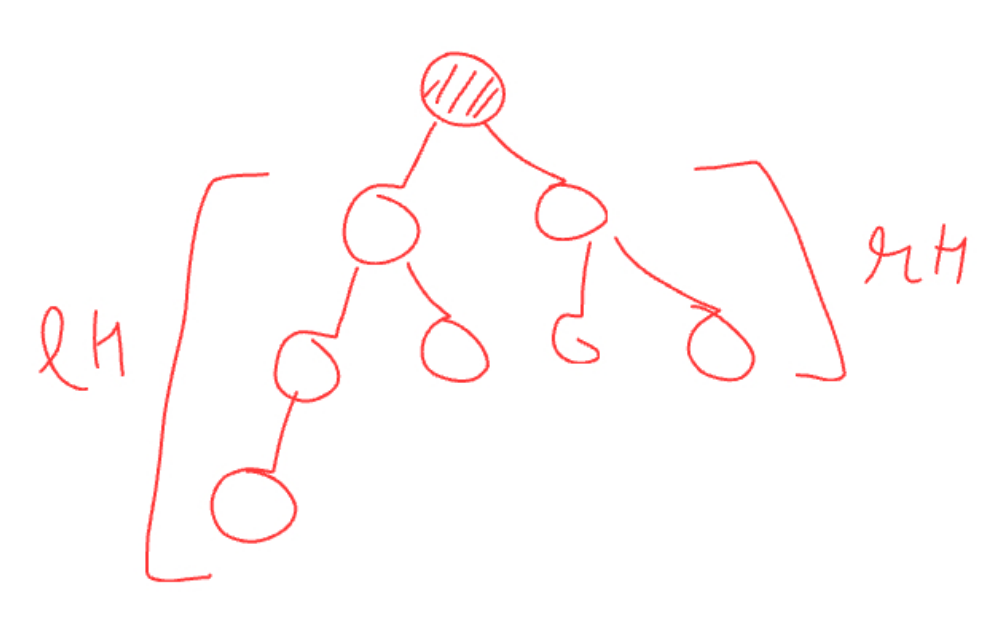
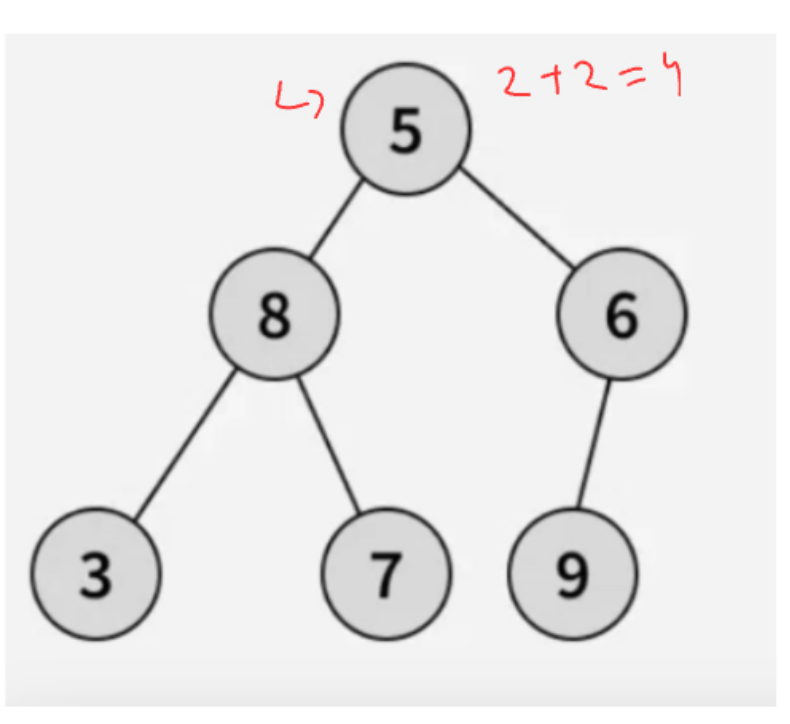
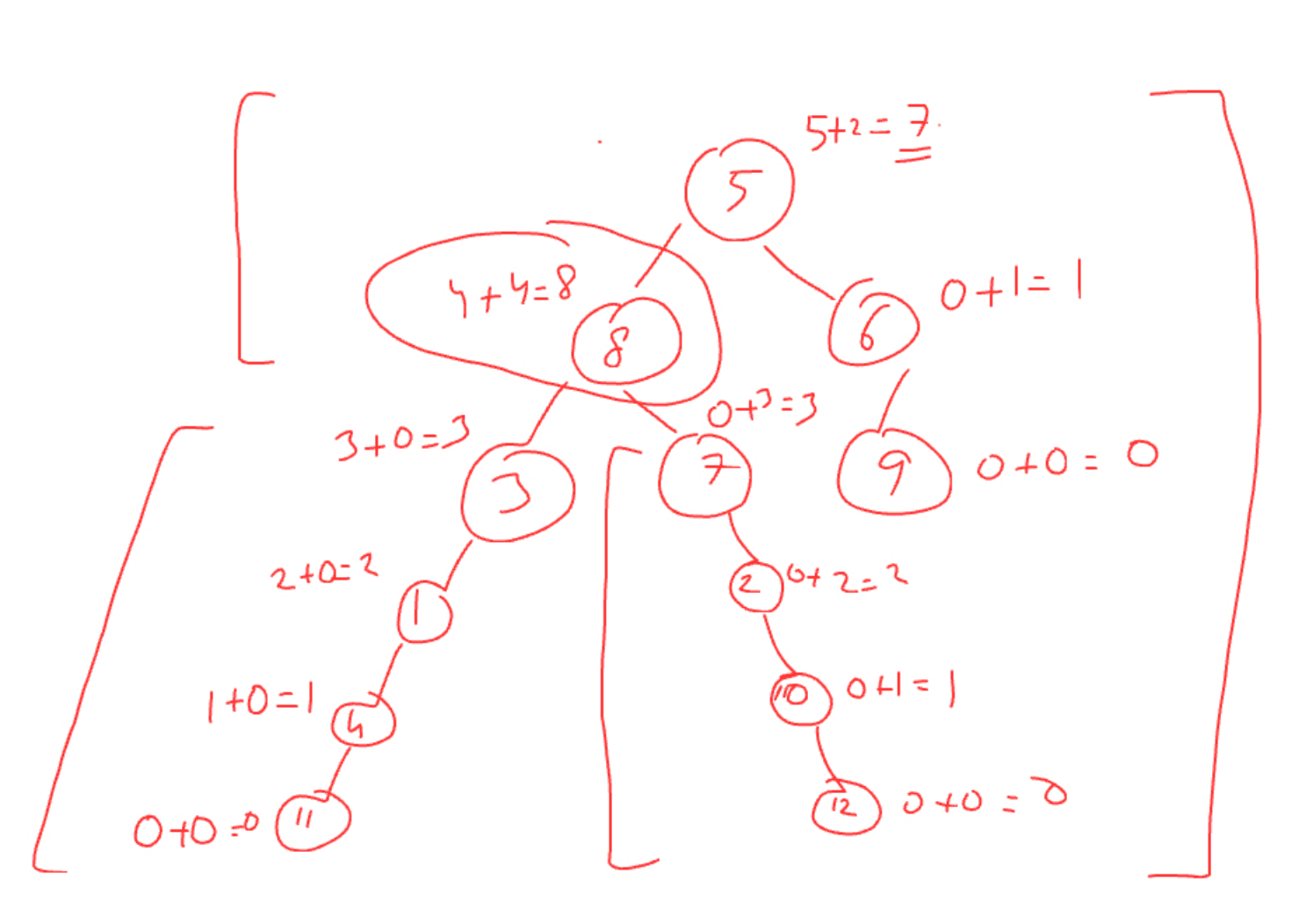


H = 1 + max(lH, rH).


4
2+2=4
5
8
6
3
7
9

4


5+1=7
5
4+4=8
8
6
0+1=1
3+0=3
0+2=3
7
9
0+0=0
2+0=2
1
2
0+2=2
1+0=1
5
0
0+1=1
0+0=0
11
12
0+0=0

(11 → 9) = 7

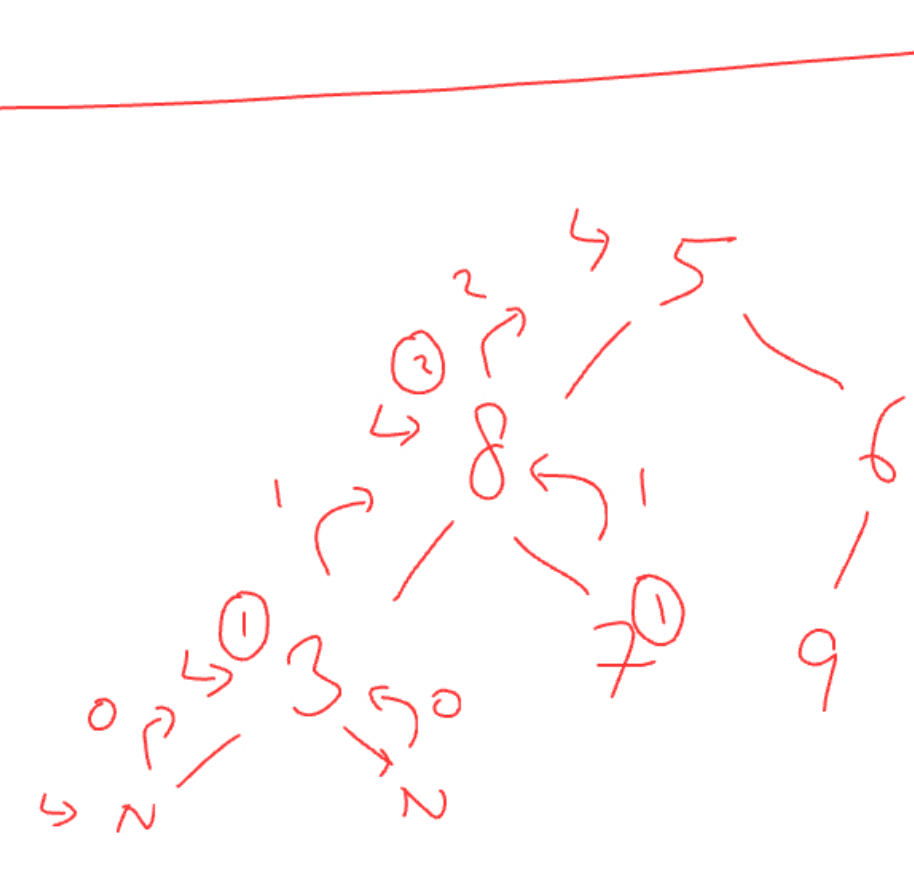
(11 → 12) = 8

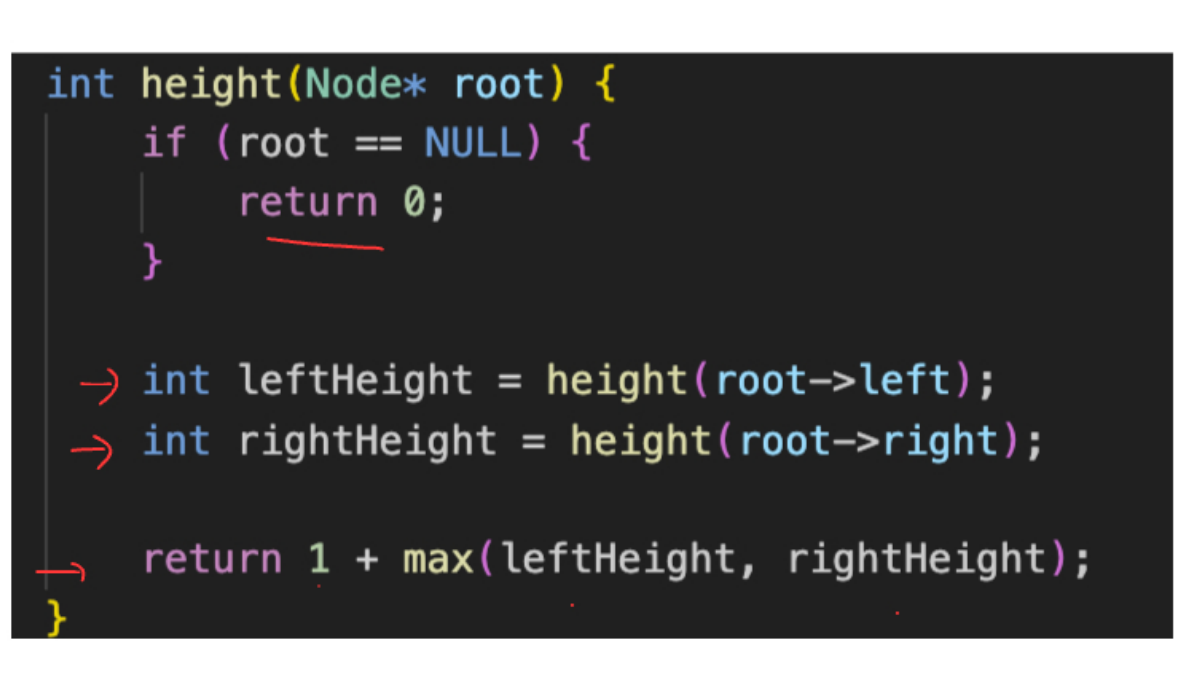
TC = O(n^n) = O(n^2)

AS = ________

---

res = 0

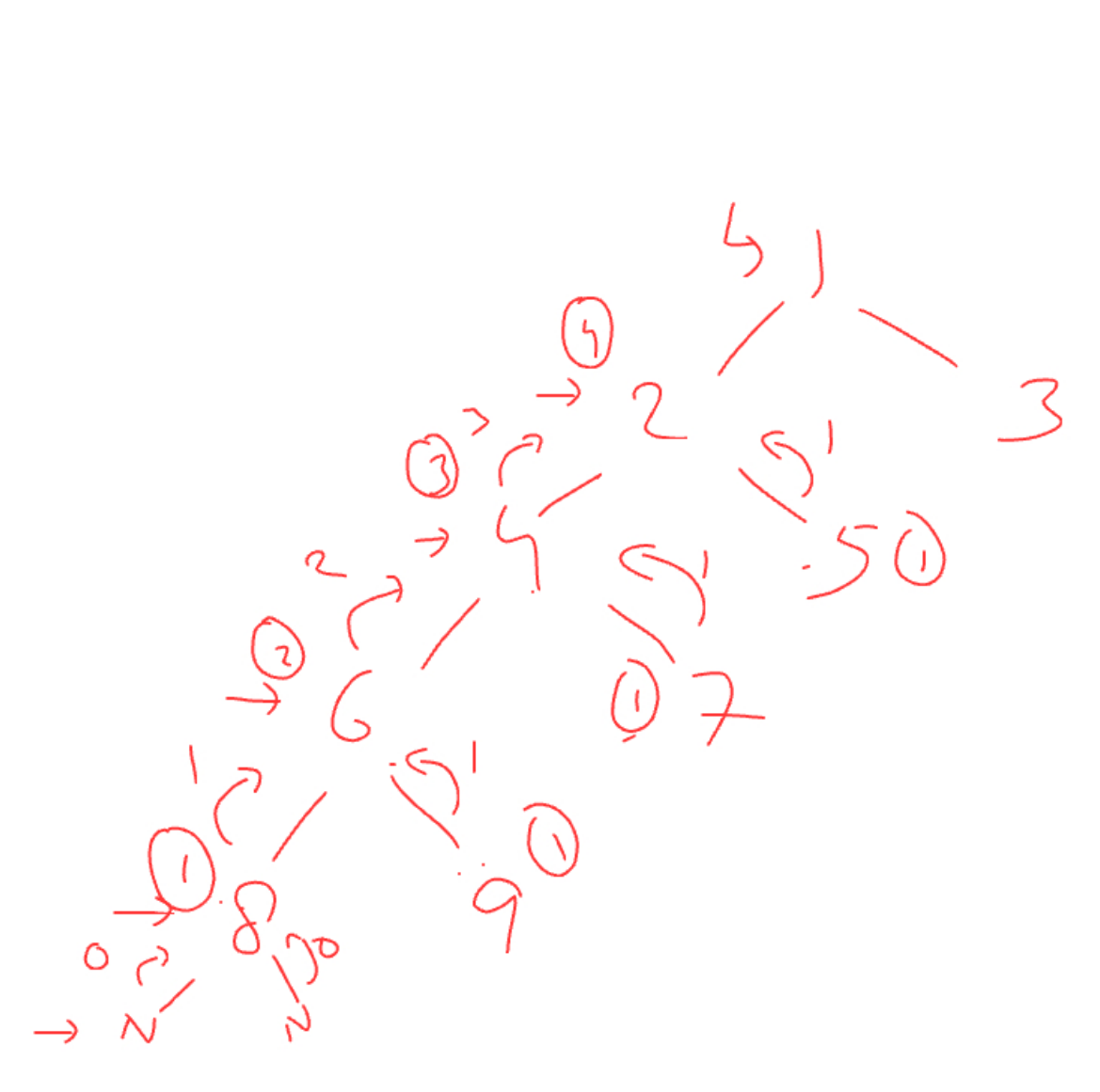




```
int height(Node* root) {
    if (root == NULL) {
        return 0;
    }

    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    return 1 + max(leftHeight, rightHeight);
}
```

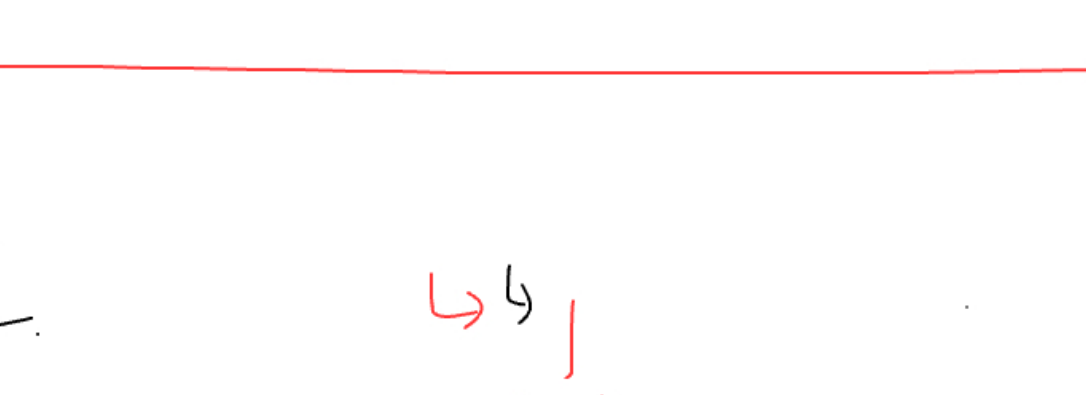

1
2
3
5
6
7
8
9
N
N

map

heights {Node* → int}

~~n1 = 4~~
~~n2 = 4~~

n1 = 6
n2 = 7



7
~~6~~
3
~~5~~
~~4~~
~~2~~
1

currPath [1 | 2 | ~~4~~ 5 | 3 | 6]

res. Path

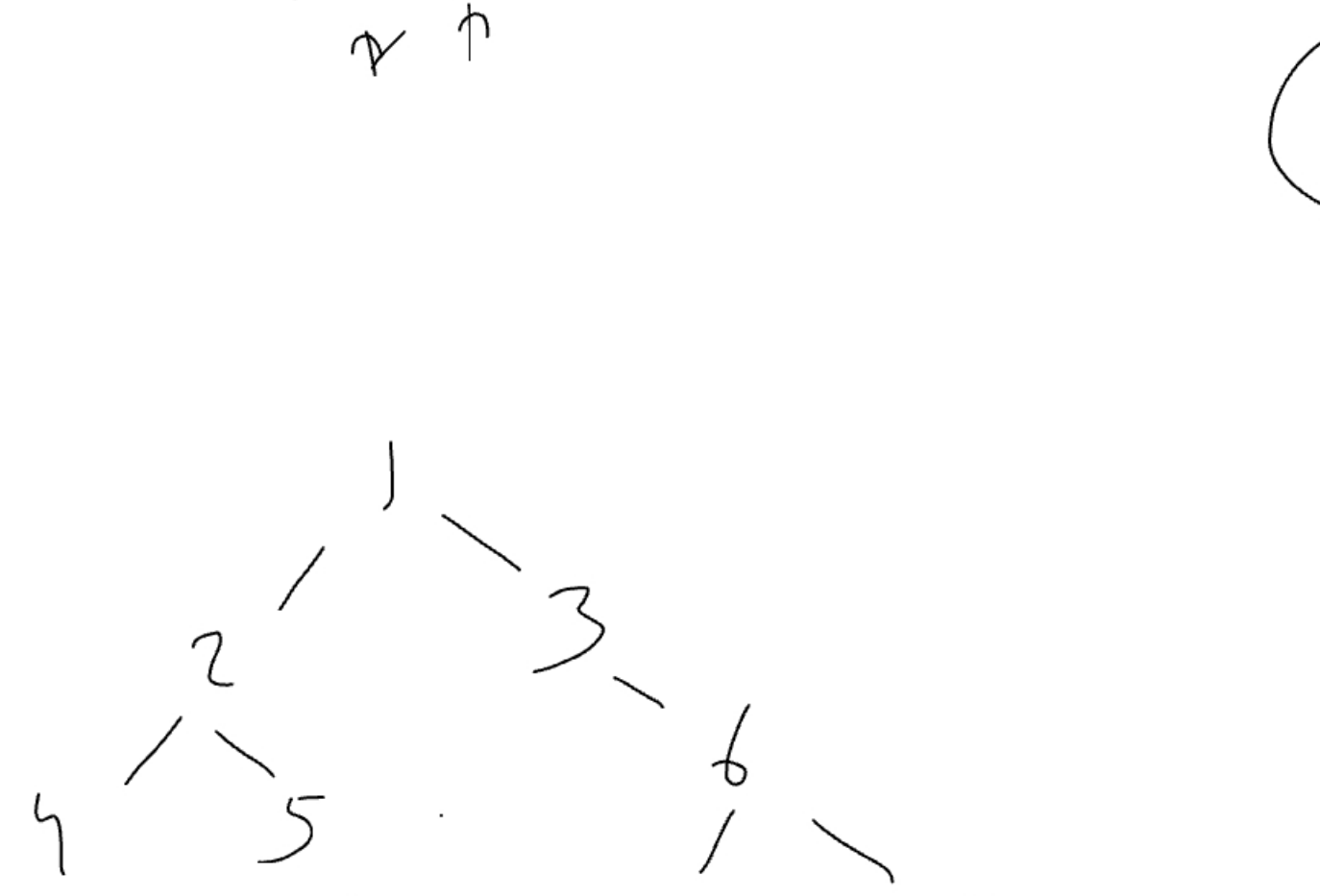
→ path6 = [1 | 3 | 6]

path7 = [1 | 3 | 7]

→ path 5 = [1 | 2 | 5]

res = 1

res = ~~1~~ 3


1
2
3
4
5
6
7
8
9
10
11

lca (4, 8) = 2
lca (2, 7) = 2
lca (10, 11) = 10
lca (7, 11) = 1

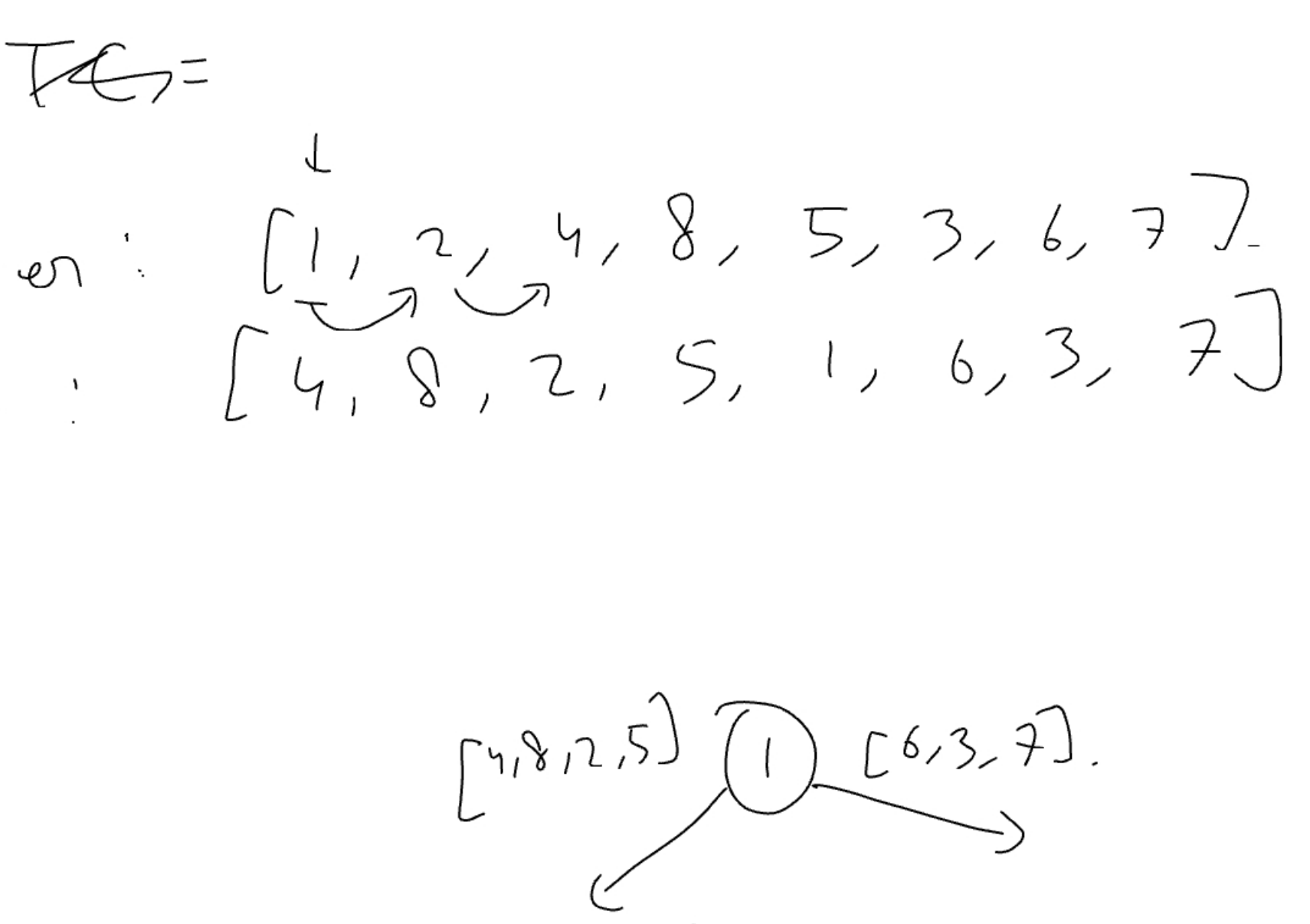
NOTE → Only a single unique Binary Tree can be formed out of given:

1) Preorder + inorder
2) Postorder + inorder

inorder: 4, 8, 2, 5, 1, 6, 3, 7. [L S R]

postorder: 8, 4, 5, 2, 6, 7, 3, 1 [L R S]

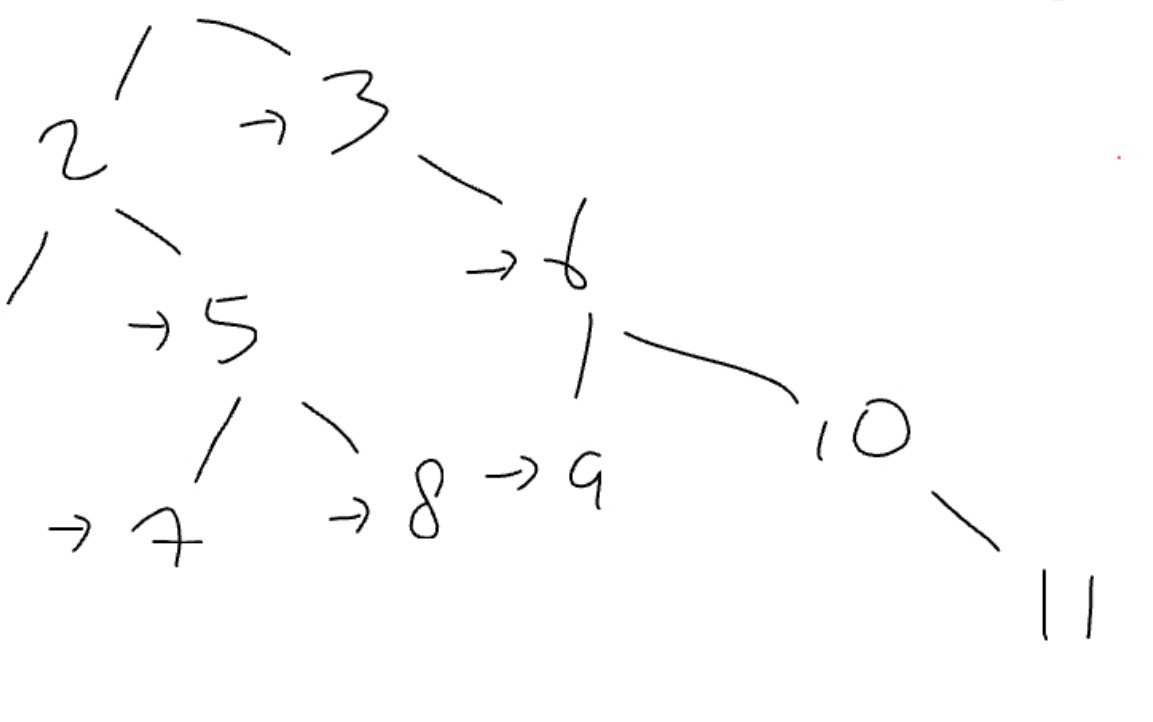
[value → index in inorder.]


[4,8,2,5]
1
[6,3,7]
[4,8]
2
[5]
3
[7]
[]
4
[8]
5
[6]
6
[]
7
[]
8
N
N
N
N
N

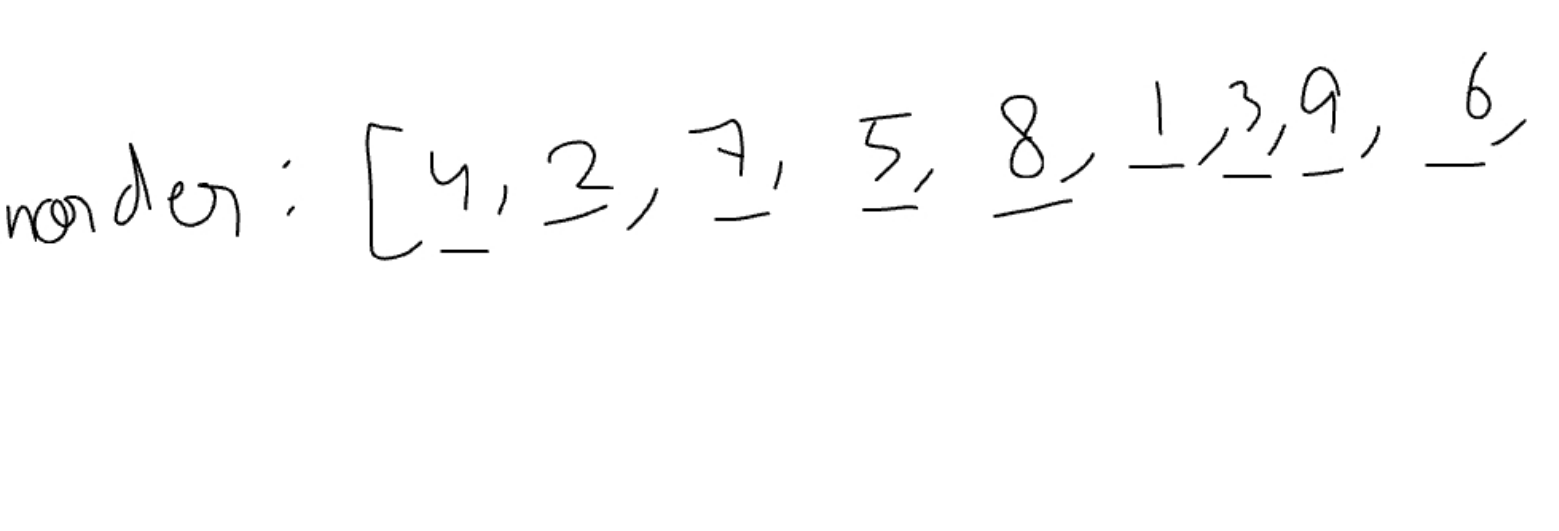
~~TC~~=

Preorder: [1, 2, 4, 8, 5, 3, 6, 7] [S L R]

inorder: [4, 8, 2, 5, 1, 6, 3, 7] [L S R].


[4,8,2,5]
1
[6,3,7].
[4,8]
2
[5]
4

[L S R].


1
2
3
4
5
6
7
8
9
10
11

inorder: [4, 2, 7, 5, 8, 1, 3, 9, 6, 10, 11].

~~11~~
~~10~~
~~9~~
~~6~~
~~3~~
~~8~~
~~7~~
~~5~~
~~4~~
~~2~~
~~1~~

~~11~~
~~10~~
~~9~~
~~6~~
~~3~~
~~8~~
~~7~~
~~5~~
~~4~~
~~2~~
~~1~~

[4, 2, 7, 5, 8, 1, 3, 9, 6, 10, 11]

TC = O(n)

AS = O(H).